

Wanderlust — Dockerfiles & Kubernetes Manifests Explained

All the code explained here is in the GitHub repository:
github.com/nitin1094/wanderlust-mern-app-k8

A Beginner's Line-by-Line Walkthrough of the Wanderlust MERN project — both Dockerfiles, the Compose file, and every manifest in `kubernetes/`, one line at a time.

What this guide covers

Backend Dockerfile	A two-stage build — and why most of its benefit is given away
Frontend Dockerfile	Layer caching done right, and a dev server shipped to production
.env.docker files	The four values that decide whether anything can reach anything
docker-compose.yml	All four services on one machine — and where it disagrees with the env files
PV & PVC	hostPath storage, and the one line that makes the claim bind correctly
Four Deployments	MongoDB, Redis, Express and React — plus the volume bug in <code>redis.yml</code>
Two NodePorts	How 31000 and 31100 pair up, and why the browser needs both
Appendices	Field reference, placeholders, error decoder, interview Q&A & glossary

Written in plain language for readers with only basic Docker and Kubernetes knowledge. Every file is quoted *verbatim* from the repository — nothing simplified, nothing invented — so you can open the real code alongside this guide and follow along. A green “From the repo:” link under each chapter heading points at the exact file.

Table of Contents

Every entry is a live link — click it to jump straight to that topic.

The App and the Files That Ship It	3
1. What This Guide Explains	3
2. Docker in Five Minutes	4
3. Kubernetes in Five Minutes	6
Building the Images (backend/, frontend/, root)	9
4. The Backend Dockerfile, Line by Line	9
5. The Frontend Dockerfile, Line by Line	12
6. The .env.docker Files	15
7. docker-compose.yml	18
The kubernetes/ Folder	21
8. How to Read These Manifests	21
9. Storage - persistentVolume.yaml and persistentVolumeClaim.yaml	24
10. mongodb.yaml	26
11. redis.yaml	30
12. backend.yaml	32
13. frontend.yaml	35
14. Putting It All Together	37
Appendices	38
15. Field Reference Cheat Sheet	38
16. Placeholders You Must Replace	41
17. Common Errors and What They Mean	42
18. Interview Questions and Answers	43
19. Glossary	45
20. Links and Repository	47

The App and the Files That Ship It

What this guide covers, the four-tier MERN app behind the files, and just enough Docker and Kubernetes to make every line that follows meaningful.

1. What This Guide Explains

This is a **code-reading guide**, not a deployment tutorial. Its job is to open every Dockerfile, the Compose file, and every Kubernetes manifest in the Wanderlust project and explain them **line by line**, in plain English, so nothing in them is a mystery to you.

You need only basic Docker and Kubernetes knowledge: roughly what a container is, and that Kubernetes runs containers for you. Everything else is explained here as it comes up.

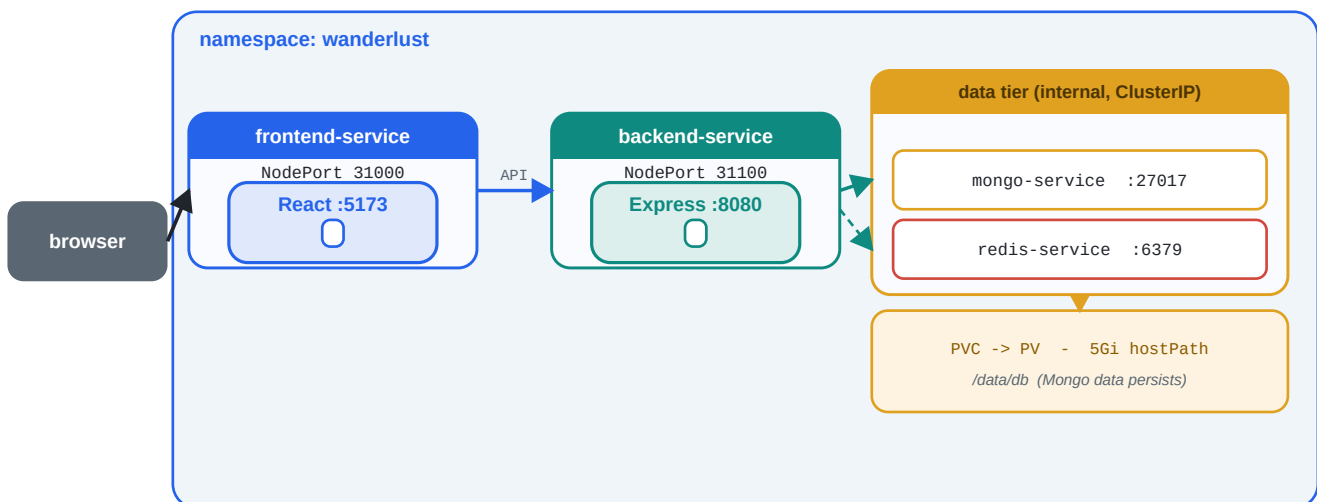
The app these files package

Wanderlust is a travel blog — you read posts, write your own, and sign in. It is a **MERN** application, and unlike a simple two-tier app it has **four** moving parts:

Tier	What it is	Technology	Listens on
Presentation	The web pages you click on	React 18 + TypeScript, served by Vite	port 5173
Application	The API behind them	Node.js + Express + Mongoose	port 8080
Data	Where posts and users are stored	MongoDB	port 27017
Cache	Keeps hot posts in memory so MongoDB is asked less often	Redis	port 6379

The MERN app on a kubeadm cluster

React frontend (NodePort 31000) -> Express backend (NodePort 31100) -> MongoDB & Redis (internal)



NOTE MERN stands for **M**ongoDB, **E**xpress, **R**eact, **N**ode.js. Redis is the fifth wheel — an optional cache the backend uses if it can reach one, and quietly skips if it cannot. Chapter 11 shows exactly how that "quietly skips" is written.

Everything is driven by environment variables

The backend reads its whole world from three variables, in `backend/config/utils.js`:

```
const PORT = process.env.PORT;
const MONGODB_URI = process.env.MONGODB_URI;
const REDIS_URL = process.env.REDIS_URL;
```

And the frontend reads exactly one, in its API client:

```
VITE_API_PATH // the public address of the backend, baked in by Vite
```

That is the whole integration surface. Every file in this guide exists to set those four values correctly for one environment or another.

KEY IDEA Whenever you see `MONGODB_URI`, `REDIS_URL`, `PORT` or `VITE_API_PATH` in a `.env.docker` file, a Compose file or a manifest, it is being fed to those lines. Get all four right and the app works; get one wrong and you will spend an evening debugging.

Where the original code lives

Every file quoted here is copied **verbatim** from the project's GitHub repository. Nothing has been rewritten or tidied — if a line looks odd, that is genuinely how it is, and this guide explains why.

From the repo: github.com/nitin1094/wanderlust-mern-app-k8

A green "**From the repo:**" link under each chapter heading points at the exact file, so you can open the real code beside this guide.

The files we will read

```
wanderlust-mern-app-k8/
+-- backend/
| +-- Dockerfile # Part II, chapter 4
| +-- .env.docker # Part II, chapter 6
+-- frontend/
| +-- Dockerfile # Part II, chapter 5
| +-- .env.docker # Part II, chapter 6
+-- docker-compose.yml # Part II, chapter 7
|
+-- kubernetes/ # PART III
    +-- persistentVolume.yaml # chapter 9
    +-- persistentVolumeClaim.yaml # chapter 9
    +-- mongodb.yaml # chapter 10 (Deployment + Service)
    +-- redis.yaml # chapter 11 (Deployment + Service)
    +-- backend.yaml # chapter 12 (Deployment + Service)
    +-- frontend.yaml # chapter 13 (Deployment + Service)
```

Six manifest files for eight Kubernetes objects — because four of them define **two objects each**. Chapter 8 explains how.

NOTE Unlike some projects, there is only **one** manifest folder here. These manifests target a self-managed cluster built with `kubeadm` — two EC2 machines, one control plane and one worker — and they expose the app with `NodePort` rather than a cloud load balancer.

2. Docker in Five Minutes

Just enough Docker to make every line of Part II meaningful.

The three levels: Dockerfile, image, container

Think of cooking:

- A **Dockerfile** is the **recipe** — a plain text file of instructions, written once.
- An **image** is the **frozen meal** made by following that recipe — a read-only, shippable bundle containing your app, its libraries, and the operating-system files it needs.
- A **container** is the **meal being eaten** — a running copy of the image. Many containers can run from one image without interfering.

`docker build` turns the recipe into the frozen meal; `docker run` serves it. In Part III, Kubernetes does the serving.

ANALOGY An image is like a class in programming; a container is like an object created from it. Write the class once, make as many objects as you like.

Images are built in layers

Every instruction (`FROM`, `COPY`, `RUN`, ...) creates one **layer** — a saved snapshot of the filesystem at that point. Layers stack to form the final image.

This matters for one practical reason: **Docker caches layers**. On a rebuild, Docker walks the file from the top and reuses each cached layer until it hits an instruction whose inputs changed. From there down, everything is rebuilt.

The two Dockerfiles in this repo order their instructions differently, and chapters 4 and 5 show what that costs.

Multi-stage builds

Both Dockerfiles here have **two** `FROM` lines, which makes them **multi-stage builds**. The idea: use a big image with all the build tools to *produce* your app, then start a fresh, smaller image and copy only the finished result into it.

Instruction	What it does
<code>FROM node:21 AS builder</code>	Starts a stage and gives it the name <code>builder</code>
<code>FROM node:21-slim</code>	A second <code>FROM</code> — everything above is discarded unless copied forward
<code>COPY --from=builder /app .</code>	Copies out of that earlier stage rather than from your machine

KEY IDEA `COPY --from=<stage>` is the line that makes a multi-stage build worth doing. Chapter 4 shows a case where the technique is used but most of the benefit is thrown away — which is a more useful thing to understand than a textbook example.

The instructions used in this project

Instruction	What it does
FROM	Picks the image to build on top of
WORKDIR	Sets the "current folder" inside the image for everything after it
COPY	Copies files from your machine into the image
RUN	Runs a command while building , saving the result as a layer
EXPOSE	Documents which port the app listens on. Opens nothing by itself.
CMD	The default command to run when a container starts — not during the build

NOTE `RUN` happens during `docker build`; `CMD` happens during `docker run`. The test: if it should be *baked into* the image, it is a `RUN`. If it should happen *fresh each time* a container starts, it is `CMD`.

3. Kubernetes in Five Minutes

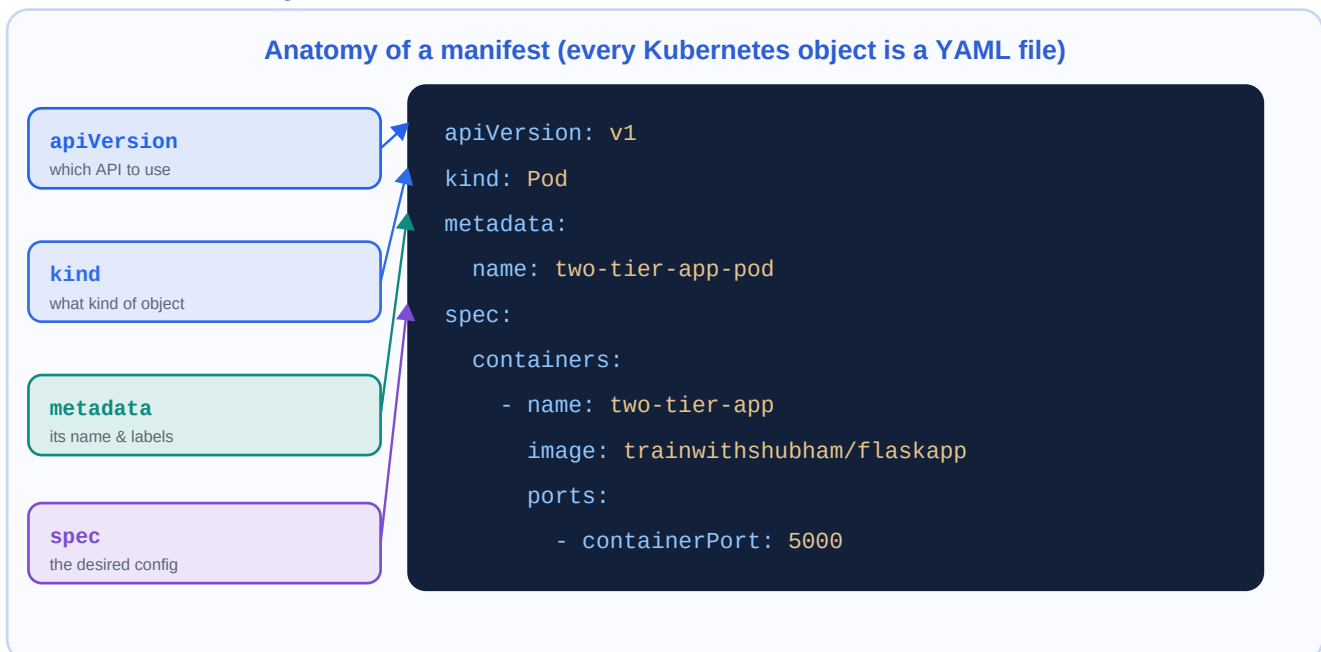
Now the vocabulary for Part III.

Everything is a described object

You never tell Kubernetes *how* to do something. You hand it a YAML file describing **what you want**, and Kubernetes works continuously to make reality match. This is called being **declarative**, and the loop that keeps reality in sync is **reconciliation**.

Read every manifest as a wish, not a script: *"I want one MongoDB pod, using this image, with this volume attached."*

The four fields every manifest starts with



Every object in `kubernetes/` opens with the same keys:

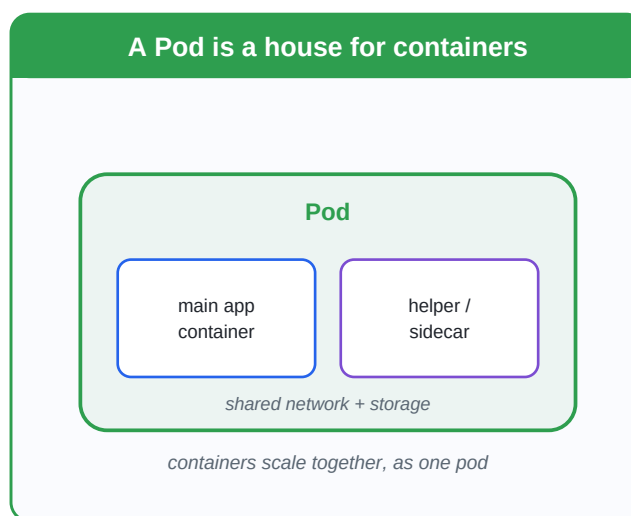
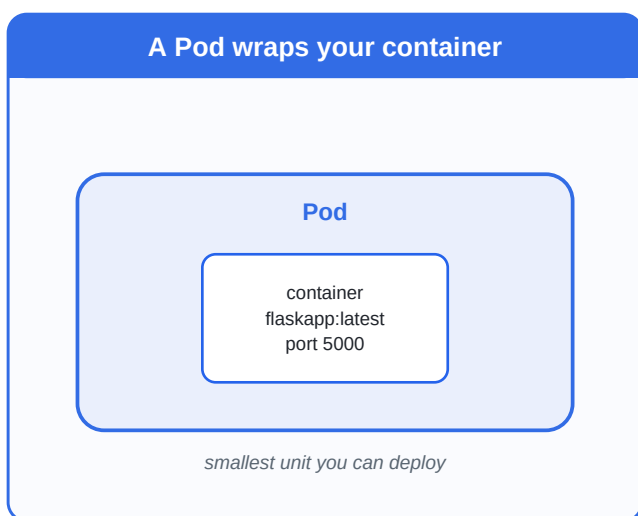
Field	Meaning	Examples in this project
<code>apiVersion</code>	Which version of the Kubernetes API this object belongs to	<code>v1</code> , <code>apps/v1</code>
<code>kind</code>	What type of object you are creating	<code>Deployment</code> , <code>Service</code> , <code>PersistentVolume</code>

Field	Meaning	Examples in this project
meta data	Its name, namespace and labels	name: mongo-deployment, namespace: wanderlust
spec	The actual wish — everything specific to that kind	replicas, image, ports, volumes...

TIP `apiVersion: v1` covers the oldest, most fundamental objects (Service, PersistentVolume, PersistentVolumeClaim). `apps/v1` covers workload controllers (Deployment). A "no matches for kind" error almost always means a wrong `apiVersion`.

The object types you will meet

Object	In one sentence
Pod	The smallest unit Kubernetes runs — one or more containers sharing a network address.
Deployment	A manager that keeps N identical pods alive and replaces them when they die.
Service	A stable name and IP in front of a changing set of pods.
PersistentVolume (PV)	Storage that exists independently of any pod, so data survives a restart.
PersistentVolumeClaim (PVC)	A pod's <i>request</i> for storage: "I need 5 GiB."
Namespace	A folder inside the cluster grouping related objects — here, <code>wanderlust</code> .



Labels and selectors — the glue

A **label** is a key/value sticker you put on an object:

```
labels:
  app: mongo # a sticker saying "this pod is the database"
```

A **selector** is a search for that sticker:

```
selector:
  app: mongo # "find me everything wearing the 'app: mongo' sticker"
```

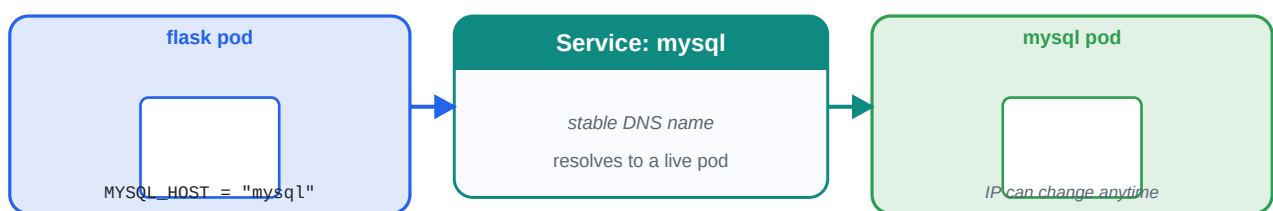
Deployments use a selector to know which pods they own. Services use one to know where to send traffic. Nothing is wired by IP address — **everything is wired by matching labels**.

Four label values do all the work here:

Label	Worn by	Searched for by
app: frontend	the React pods	frontend-deployment and frontend-service
app: backend	the Express pods	backend-deployment and backend-service
app: mongo	the MongoDB pod	mongo-deployment and mongo-service
app: redis	the Redis pod	redis-deployment and redis-service

WATCH OUT If a Service's selector does not exactly match the labels on the pods, the Service has **no endpoints** and traffic silently goes nowhere — with no error message. When something "just doesn't connect", check `kubectl get endpoints` before anything else.

Service discovery — the trick that ties the tiers together



Never hard-code a pod's IP. Point to the service NAME and Kubernetes routes to a healthy pod.

The backend is told `MONGODB_URI="mongodb://mongo-service/wanderlust"`. That hostname, `mongo-service`, is not something you configured in DNS — it is the **name of the MongoDB Service**. Kubernetes runs an internal DNS server that turns every Service name into its virtual IP automatically.

So the backend connects to `mongo-service`, Kubernetes resolves it, and the Service forwards to whichever pod currently matches `app: mongo`. That pod can be deleted and recreated with a brand-new IP a hundred times; the name never changes.

The same applies to `redis://redis-service:6379`.

KEY IDEA Pods are disposable and their IPs change constantly. Services exist so that nothing in your application ever has to care. This is the single most important idea in Part III — and it is why chapter 6's `.env.docker` files contain Kubernetes Service names rather than IP addresses.

The namespace: `wanderlust`

Every object in `kubernetes/` sets the same one:

```

metadata:
  namespace: wanderlust  # written into all six files

```

Because the namespace is written *into the files*, it must already exist. **No manifest in the repo creates it**, so you must do it yourself before applying anything:

```

kubectl create namespace wanderlust  # must exist first
kubectl config set-context --current --namespace wanderlust  # so you can drop -n from every command

```

That second line is why the commands in Part III carry no `-n wanderlust` flag — it makes `wanderlust` the default for your session.

WATCH OUT Applying these manifests without creating the namespace first fails with `namespaces "wanderlust" not found`. It is the very first thing that goes wrong for most people, and the error message is at least honest about it.

PART II

Building the Images (backend/, frontend/, root)

Both Dockerfiles line by line, the two `.env.docker` files that decide every address, and the Compose file that runs all four services on one machine.

4. The Backend Dockerfile, Line by Line

From the repo: [backend/Dockerfile](#)

The Express API's image. It is a two-stage build, and reading it carefully teaches more than a textbook example would — because it shows the technique used in a way that gives up most of its benefit.

```
# Stage 1
FROM node:21 AS backend-builder

# setup the working dir
WORKDIR /app

# code
COPY . .

# packages install
RUN npm i

# tests
RUN npm run test

# Stage 2
FROM node:21-slim

# setup the working dir
WORKDIR /app

# copy the above stage as compressed
COPY --from=backend-builder /app .

COPY .env.docker .env

# Port
EXPOSE 8080

# App
CMD ["npm", "start"]
```

Stage 1 — FROM node:21 AS backend-builder

```
FROM node:21 AS backend-builder # the FULL Node 21 image, named "backend-builder"
WORKDIR /app # cd into /app; create it if missing
```

`FROM` picks the **base image** — here the official Node.js 21 image, which already contains a Debian filesystem, Node and `npm`. `AS backend-builder` gives this stage a **name**, so a later stage can copy out of it. `WORKDIR` is Docker's `cd`, with a bonus: it creates the folder if it is missing. Everything after it is relative to `/app`.

NOTE Node 21 was a short-lived "current" release and reached end of life in **June 2024**. Even-numbered Node releases (20, 22, 24) get long-term support; odd ones do not. For anything real, move to `node:22-slim` and pin it.

`COPY . .` then `RUN npm i` — the wrong way round

```
COPY . . # copy the ENTIRE source tree first
RUN npm i # ...then install dependencies
```

This ordering throws away Docker's layer cache, and it is worth understanding exactly why.

Docker reuses a cached layer only while that instruction's inputs are unchanged. Because `COPY . .` takes in **every file in the backend folder**, editing a single line of `server.js` changes that layer — and therefore invalidates `RUN npm i` below it. Every code change reinstalls every dependency from scratch.

The frontend Dockerfile in chapter 5 gets this right, and the contrast is the clearest lesson in the repo:

	Backend (this file)	Frontend (chapter 5)
Order	<code>COPY . .</code> then <code>npm i</code>	<code>COPY package*.json ./</code> , <code>npm install</code> , then <code>COPY . .</code>
Editing one source file	Reinstalls all dependencies	Reuses the cached install
Typical rebuild	Minutes	Seconds

KEY IDEA Order a Dockerfile from **least likely to change** at the top to **most likely to change** at the bottom. Dependency manifests change rarely; source code changes constantly. So copy `package*.json`, install, and only then copy the source. Swapping two lines in this file would fix it: `` `COPY package*.json ./` `RUN npm i` `COPY . .` ``

WATCH OUT There is **no** `.dockerignore` in `backend/`, so `COPY . .` also copies `.git`, your local `node_modules`, `coverage/`, and any `.env` you have lying around. A local `node_modules` built on Windows or macOS can contain native modules compiled for the wrong platform, and `npm i` afterwards will not necessarily replace them. Adding a `.dockerignore` is the single cheapest fix in this repo.

`RUN npm run test` — tests inside the build

```
RUN npm run test # run the Jest suite as part of the image build
```

Running tests during `docker build` is a real technique: if the tests fail, the `RUN` returns a non-zero exit code and the build stops, so a broken image is never produced.

The catch is what that script actually is, in `backend/package.json`:

```
"test": "jest --watchAll --detectOpenHandles --verbose --coverage "
```

`--watchAll` is Jest's **watch mode** — it runs the suite and then *keeps running*, waiting for file changes. That is exactly what you want on your laptop and exactly what you do not want in a build, which has no one to press `q`.

What rescues it is a small file the repo ships, `backend/tests/teardown.js`:

```
export default () => {
  process.exit(0);
};
```

`package.json` registers it as Jest's `globalTeardown`, so when Jest finishes its run it calls this function, which force-exits the process with a success code — and the build moves on.

WATCH OUT That is a fragile arrangement. It depends on a teardown hook force-killing the test runner to escape a watch mode that should not have been requested. If your build ever hangs at `RUN npm run test`, this is why. The robust fix is to stop asking for watch mode in the first place: `"test": "jest --ci --watchAll=false --detectOpenHandles --coverage"`, and keep a separate `"test:watch"` script for local work.

TIP Note also that `--coverage` writes a `coverage/` directory into the image, and it is carried forward into the final stage by the copy below. Test artefacts have no business in a production image.

Stage 2 — and the copy that gives the game away

```
FROM node:21-slim # a SECOND FROM: stage 1 is now discarded
WORKDIR /app
COPY --from=backend-builder /app . # ...except this, which brings ALL of it back
```

A second `FROM` starts a fresh image. `node:21-slim` is the same Node on a trimmed Debian base — roughly 250 MB against the full image's 1.1 GB.

Then `COPY --from=backend-builder /app .` copies **the entire `/app` directory** out of stage 1: the source, `node_modules`, the tests, the coverage report, everything.

So what did the multi-stage build actually buy? Only the smaller **base image**. The comment in the file — *"copy the above stage as compressed"* — is optimistic; nothing is compressed, and nothing is left behind.

A genuine slimming would install production dependencies only and copy just what runs:

```
# stage 1 additionally:
RUN npm ci --omit=dev # production dependencies only, reproducible from the lock file

# stage 2 instead of copying everything:
COPY --from=backend-builder /app/node_modules ./node_modules
COPY --from=backend-builder /app/server.js /app/api /app/config /app/routes /app/models ./
```

KEY IDEA A multi-stage build only helps if the final stage copies **less** than the builder produced. `COPY --from=builder /app .` copies all of it, so the technique is present but the benefit is not. This is a common pattern to recognise in real repositories.

`COPY .env.docker .env` — configuration baked into the image

```
COPY .env.docker .env # rename the docker env file to the name dotenv looks for
```

The backend loads configuration with the `dotenv` package, which reads a file literally named `.env`. The repo keeps that file out of version control (it is in `.gitignore`) and ships `.env.docker` instead, so this line renames it on the way in.

WATCH OUT This **bakes configuration into the image**, including `JWT_SECRET`. Anyone who can pull your image can read it with `docker run --rm <image> cat .env`. It also means one image per environment — change a URL and you must rebuild and re-push. In Kubernetes the right answer is a `ConfigMap` for the addresses and a `Secret` for the JWT key, injected as environment variables at run time. Chapter 12 comes back to this, because the backend manifest sets **no** environment variables at all — it relies entirely on this baked-in file.

`EXPOSE 8080` **and** `CMD ["npm", "start"]`

```
EXPOSE 8080 # documentation: "this app listens on 8080"
CMD ["npm", "start"] # what to run when a container starts
```

`EXPOSE` **opens nothing**. It is metadata — a note for humans and for `docker run -P`. The port is reachable whether or not you declare it. What actually decides the port is `PORT=8080` in `.env.docker` (chapter 6), read by `server.js`:

```
const port = PORT || 5000; // PORT comes from the environment
app.listen(port, () => console.log(`Server is running on port ${port}`));
```

So: no `PORT` set, and the backend falls back to **5000**. That fallback matters in chapter 7.

`CMD ["npm", "start"]` runs the `start` script, which is:

```
"start": "nodemon server.js"
```

WATCH OUT `nodemon` is a **development** tool. It watches your source files and restarts the app whenever one changes — useful on a laptop, pointless in a container whose files never change, and it adds a file-watcher and a supervisor process you do not want in production. It also means the app is not PID 1, so shutdown signals are handled less cleanly. `CMD ["node", "server.js"]` is the production form.

The backend Dockerfile at a glance

Step	Instruction	Runs when?	Note
1	<code>FROM node:21 AS backend-builder</code>	build	Full image, named stage
2	<code>WORKDIR /app</code>	build	Working folder
3	<code>COPY . .</code>	build	Before install — defeats the cache
4	<code>RUN npm i</code>	build	Re-runs on every code change
5	<code>RUN npm run test</code>	build	Escapes watch mode only via <code>globalTeardown</code>
6	<code>FROM node:21-slim</code>	build	Fresh, smaller base
7	<code>COPY --from=backend-builder /app .</code>	build	Copies everything back
8	<code>COPY .env.docker .env</code>	build	Config baked into the image
9	<code>EXPOSE 8080</code>	build	Documentation only
10	<code>CMD ["npm", "start"]</code>	run	<code>nodemon server.js</code>

5. The Frontend Dockerfile, Line by Line

From the repo: [frontend/Dockerfile](#)

The React image. Same two-stage shape, but the dependency ordering is done properly.

```
# ----- Stage 1: Build Stage -----
FROM node:21 AS frontend-builder

# Set the working directory to /app
WORKDIR /app

# Copy the package.json and package-lock.json for dependency installation
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# ----- Stage 2: Final Stage -----
FROM node:21-slim

# Set the working directory to /app
WORKDIR /app

# Copy built assets and dependencies from frontend-builder stage
COPY --from=frontend-builder /app .

# Copy the .env.sample file to .env.local
COPY .env.docker .env.local

# Expose port 5173 for the Node.js application
EXPOSE 5173

# Define the default command to run the application in development mode
CMD ["npm", "run", "dev", "--", "--host"]
```

The dependency ordering, done right

```
COPY package*.json ./ # copy ONLY the dependency manifests
RUN npm install # install them - this layer now caches well
COPY . . # THEN the source code
```

This is the pattern the backend is missing. `package*.json` is a wildcard matching both `package.json` and `package-lock.json`. Those two small files are copied alone, dependencies are installed, and only then does the application source arrive.

You changed...	<code>COPY package*.json</code>	<code>RUN npm install</code>	<code>COPY . .</code>
A component in <code>src/</code> (typical)	cache hit	cache hit — skipped entirely	rebuilt (fast)
<code>package.json</code> (added a library)	cache miss	re-runs (slow, but necessary)	rebuilt

Since the frontend has around 50 dependencies, the difference between a cached and an uncached install is the difference between a two-second rebuild and a two-minute one.

WATCH OUT As with the backend, there is **no** `.dockerignore` in `frontend/`. `COPY . .` will happily copy your local `node_modules` straight over the ones `npm install` just created in the image. Add one containing at least `node_modules`, `.git`, `dist` and `.env.local`.

Stage 2 — same copy-everything pattern

```
FROM node:21-slim
WORKDIR /app
COPY --from=frontend-builder /app . # again: the whole tree, node_modules included
```

Identical to the backend: the final image gets a slimmer *base*, but all of stage 1's contents come across.

Here, though, there is a reason — and it is the biggest design decision in this file.

CMD ["npm", "run", "dev", "--", "--host"] — the dev server in production

```
CMD ["npm", "run", "dev", "--", "--host"]
```

Looking at `frontend/package.json`:

```
"dev": "vite",
"build": "tsc -p tsconfig.prod.json && vite build",
"preview": "vite preview"
```

So the container runs `vite` — the **Vite development server**. It compiles TypeScript and React on the fly, serves the app on port 5173, and watches for file changes to hot-reload the browser. That is why `node_modules` had to be copied forward: the dev server needs the whole toolchain at run time.

The `--` in the middle is npm syntax meaning *"everything after this goes to the underlying command, not to npm"*. So `--host` reaches Vite, where it means **bind to 0.0.0.0** instead of `localhost`.

KEY IDEA `--host` is not optional here. By default Vite listens only on `127.0.0.1`, which accepts connections from inside the container alone. The port mapping would work and nothing would ever answer. Any server in a container needs to bind `0.0.0.0` — the same reason the two-tier Flask app in the companion guide calls `app.run(host='0.0.0.0')`.

WATCH OUT Shipping a dev server is the biggest compromise in this project. `vite` is slow to boot, holds the whole TypeScript toolchain in memory, serves unminified code, and is explicitly documented as not for production use. The app works, but it is not how you would serve real users.

What a production frontend image looks like

The proper pattern compiles the app in stage 1 and serves the resulting static files from a tiny web server. This is **not** in the repo — it is the upgrade path:

```
# ----- stage 1: compile -----
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci # reproducible install from the lock file
COPY . .
ARG VITE_API_PATH # Vite bakes env vars in AT BUILD TIME
ENV VITE_API_PATH=$VITE_API_PATH
RUN npm run build # produces /app/dist - plain HTML, CSS, JS

# ----- stage 2: serve -----
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html # take ONLY the compiled output
EXPOSE 80
```

The compilers, the TypeScript toolchain and `node_modules` all stay behind in stage 1. The image drops from roughly 1 GB to about 25 MB and starts instantly.

WATCH OUT Note the `ARG` in that example. Vite replaces `import.meta.env.VITE_*` with literal strings **when it compiles**, so with `npm run build` the API address must be supplied at build time via `--build-arg`, not as a container environment variable. Because the repo's image runs `npm run dev` instead — which compiles when the container starts — copying `.env.docker` at build time happens to work. Switch to a production build without understanding this and your frontend will call `undefined/api/posts`.

```
COPY .env.docker .env.local
```

```
COPY .env.docker .env.local # Vite reads .env.local
```

Vite looks for files named `.env`, `.env.local` and so on. The repo keeps the real one out of Git and ships `.env.docker`, so this line renames it on the way in. Its single variable, `VITE_API_PATH`, is chapter 6.

NOTE The comment above that line in the repo says "Copy the `.env.sample` file to `.env.local`", but the command copies `.env.docker`. The comment is stale; the command is what runs. Worth noticing, because a wrong comment costs more debugging time than no comment.

6. The `.env.docker` Files

From the repo: [backend/.env.docker](#)

Two small files, and between them they decide whether anything in this project can talk to anything else. They are baked into the images by the two `COPY` lines you just read.

[backend/.env.docker](#)

```

MONGODB_URI="mongodb://mongo-service/wanderlust"
REDIS_URL="redis://redis-service:6379"
PORT=8080
FRONTEND_URL="http://<worker-node-public-ip>:5173"
ACCESS_COOKIE_MAXAGE=120000
ACCESS_TOKEN_EXPIRES_IN='120s'
REFRESH_COOKIE_MAXAGE=120000
REFRESH_TOKEN_EXPIRES_IN='120s'
JWT_SECRET=<your-strong-random-secret> # e.g. generate with: openssl rand -hex 64
NODE_ENV=Development

```

Line by line:

Variable	What it does
<code>MONGODB_URI</code>	Where MongoDB is. <code>mongo-service</code> is the Kubernetes Service name from chapter 10; <code>/wanderlust</code> is the database name.
<code>REDIS_URL</code>	Where Redis is — <code>redis-service</code> from chapter 11, on its standard port.
<code>PORT</code>	The port Express listens on. This is what makes it 8080 rather than the 5000 fallback.
<code>FRONTEND_URL</code>	Intended for CORS — but see the warning below.
<code>ACCESS_COOKIE_MAXAGE</code>	How long the access cookie lives, in milliseconds (120000 = 2 minutes).
<code>ACCESS_TOKEN_EXPIRES_IN</code>	How long the access JWT is valid — <code>120s</code> , matching the cookie.
<code>REFRESH_COOKIE_MAXAGE</code>	Same for the refresh cookie.
<code>REFRESH_TOKEN_EXPIRES_IN</code>	Same for the refresh token.
<code>JWT_SECRET</code>	The key used to sign and verify login tokens.
<code>NODE_ENV</code>	Conventionally <code>development</code> or <code>production</code> .

KEY IDEA `mongo-service` and `redis-service` are not hostnames anyone configured — they are **Kubernetes Service names**, resolved by the cluster's internal DNS. This file is written for Kubernetes, which is exactly why it causes trouble under Docker Compose in chapter 7.

Three variables that do nothing

Reading the backend source rather than trusting the file turns up something useful:

- `FRONTEND_URL` appears in no `.js` file in the project. `server.js` calls `app.use(cors())` with no options, which allows **every** origin. The variable is aspirational.
- `NODE_ENV` is never read either. It is also capitalised as `Development`, where Node's convention is lowercase `production` / `development`.
- `.env.sample` in the same folder lists only `MONGODB_URI` and `REDIS_URL`, so it is a poor guide to what the app now needs.

TIP This is a habit worth copying: when a config file lists a variable, `grep` the source for it before assuming it matters. `grep -rn "FRONTEND_URL" --include=*.js` . returning nothing is a complete answer in two seconds.

The placeholders, and a security note

```
JWT_SECRET=<your-strong-random-secret> # e.g. generate with: openssl rand -hex 64
FRONTEND_URL="http://<worker-node-public-ip>:5173"
```

Both are placeholders you must replace. The comment tells you how to produce a real secret:

```
openssl rand -hex 64 # 128 hex characters of cryptographic randomness
```

WATCH OUT `JWT_SECRET` signs every login token. Anyone who learns it can forge a token for any user. Because chapter 4's `COPY .env.docker .env` bakes this file into the image, pushing that image to a public Docker Hub repository publishes your secret. Keep the images private, or — much better — move the secret to a Kubernetes `Secret` and inject it as an environment variable at run time.

NOTE A subtle one. `backend/utils/cookie_options.js` sets `secure: true` and `sameSite: 'none'` on both auth cookies, which browsers only accept over **HTTPS**. Since this deployment serves plain HTTP on `http://<node-ip>:31000`, the browser will refuse to store them and logins will not persist. Browsing and reading posts work fine; signing in does not. That is a property of the app, not of the manifests, but it explains a confusing symptom.

frontend/.env.docker

From the repo: [frontend/](#) (see the warning below about `.env.docker`)

One line:

```
VITE_API_PATH="http://<worker-node-public-ip>:31100"
```

This is the most commonly misunderstood value in the whole project, so it is worth slowing down.

You might expect it to be `http://backend-service:8080` — the in-cluster Service name, like the backend uses for MongoDB. **It must not be.** The reason is *who does the calling*.

The React code does not run on the server. It is downloaded and executed **in the user's browser**, out on the public internet. That browser has never heard of your cluster's internal DNS, so `backend-service` means nothing to it. The address must be one the browser can reach: the worker node's public IP, on the backend's **NodePort**.

Call	Made by	Address used	Why
Frontend to backend	The user's browser	<code>http://<node-ip>:31100</code> — public	The browser is outside the cluster
Backend to MongoDB	The backend pod	<code>mongodb://mongo-service</code> — internal	The pod is inside the cluster
Backend to Redis	The backend pod	<code>redis://redis-service:6379</code> — internal	Same

KEY IDEA "The frontend talks to the backend" is misleading. The *frontend pod* never talks to the backend at all — it hands JavaScript to a browser, and the **browser** calls the backend. That is why frontend URLs are public addresses while backend-to-database URLs are internal Service names. Once this clicks, a whole class of confusion disappears.

WATCH OUT `31100` must match the `nodePort` in `kubernetes/backend.yaml` (chapter 12), and `<worker-node-public-ip>` must be the node you actually browse to. Change one without the other and the page loads but no posts appear — with a CORS or connection error visible only in the browser's developer console.

The one that is not in the repository

There is a catch with this file, and it will stop a fresh clone dead.

`frontend/.gitignore` contains a line reading `.env*`, which matches `.env.docker`. So `frontend/.env.docker` **is never committed** — it exists only on the machine where it was created. Its backend counterpart *is* committed, because `backend/` has no such rule.

That matters because `frontend/Dockerfile` ends with:

```
COPY .env.docker .env.local
```

`COPY` fails when the source file is missing, so on a freshly cloned repository the frontend image build stops with an error like *"failed to compute cache key: .env.docker not found"*.

WATCH OUT Clone the repo and `docker build` the frontend, and it **will fail** until you create `frontend/.env.docker` yourself. One line is all it needs: `VITE_API_PATH="http://<worker-node-public-ip>:31100" ``` Putting your own IP in it is a step you have to take anyway, so the practical impact is small — but the error message does not tell you that the file is simply absent, which makes it a confusing first five minutes.

TIP Two ways to make this less surprising if it were your repo: commit a `frontend/.env.docker.sample` (which `.env*` also matches, so it needs a `!` negation rule in `.gitignore`), or drop the `COPY` from the Dockerfile entirely and pass the value as a build argument instead.

7. docker-compose.yml

From the repo: [docker-compose.yml](#)

Before Kubernetes, there is a simpler way to run all four services together: **Docker Compose**.

```
version: "3.8"
services:
  mongodb:
    container_name: mongo
    image: mongo:latest
    volumes:
      - ./backend/data:/data
    ports:
      - "27017:27017"

  backend:
    container_name: backend
    build: ./backend
    env_file:
      - ./backend/.env.docker
    ports:
      - "5000:5000"
    depends_on:
```

```
- mongodb

frontend:
  container_name: frontend
  build: ./frontend
  env_file:
    - ./frontend/.env.docker
  ports:
    - "5173:5173"

redis:
  container_name: redis
  restart: unless-stopped
  image: redis:7.0.5-alpine
  expose:
    - 6379
  depends_on:
    - mongodb

volumes:
  data:
```

The shape of the file

Key	What it holds
version: "3.8"	The Compose file format version
services:	One block per container — here four of them
volumes:	Named volumes Compose should manage

NOTE `version:` is obsolete in modern Docker Compose (v2) and you may see a warning about it. Harmless — Compose works the format out itself, and you can delete the line.

The keys used here

Key	Meaning
<code>container_name:</code>	Fixes the container's name, which is also its hostname on the Compose network
<code>image:</code>	Pull a ready-made image
<code>build: ./backend</code>	Build from the Dockerfile in that folder instead — this is what makes chapters 4 and 5 run
<code>env_file:</code>	Load environment variables from a file
<code>ports:</code> <code>"host:container"</code>	Publish a port to your machine
<code>expose:</code>	Make a port reachable to other containers only , not to your machine
<code>volumes:</code> <code>"host:container"</code>	A bind mount
<code>depends_on:</code>	Start order
<code>restart:</code> <code>unless-stopped</code>	Restart the container unless you stopped it deliberately

Compose creates a default network for the project automatically, and every service joins it — which is why no `networks:` key is needed and why containers can reach each other by name.

Where this file and `.env.docker` disagree

Here is the thing to understand about running this project with Compose: **the `.env.docker` files are written for Kubernetes**, and Compose loads them verbatim.

WATCH OUT Three concrete mismatches follow from that. **1. The database hostname.** `.env.docker` says `mongodb://mongo-service/wanderlust`, but there is no container called `mongo-service` here — the service key is `mongodb` and its `container_name` is `mongo`. The name does not resolve, so the backend cannot reach the database. **2. The Redis hostname.** Same story: `redis://redis-service:6379` against a container named `redis`. **3. The published port.** `.env.docker` sets `PORT=8080`, so Express listens on 8080 inside the container — but Compose publishes `"5000:5000"`. Port 5000 inside the container has nothing on it, so `http://localhost:5000` refuses the connection. (5000 is the fallback in `server.js` when `PORT` is unset, which is presumably where the number came from.)

To run this stack with Compose, override the three values — for example with a `backend/.env.compose` file pointed at by `env_file:`, containing `MONGODB_URI="mongodb://mongo/wanderlust"`, `REDIS_URL="redis://redis:6379"` and `PORT=5000`. Or keep `PORT=8080` and publish `"8080:8080"` instead. The frontend needs `VITE_API_PATH="http://localhost:8080"` to match.

KEY IDEA A single `.env` file cannot serve two different runtimes whose service names differ. This is precisely the problem Kubernetes `ConfigMaps` exist to solve: keep the image free of environment-specific values and inject them per environment at run time.

The MongoDB service

```
image: mongo:latest # no version pin
volumes:
  - ./backend/data:/data # keep the database files on your machine
ports:
  - "27017:27017" # published to your machine
```

The bind mount is what makes data survive `docker compose down`. MongoDB stores its files in `/data/db`, which sits inside the mounted `/data`, so it works.

WATCH OUT `mongo:latest` is a moving tag — two builds a month apart can run different major versions, and a MongoDB major upgrade can refuse to open an older data directory. Pin it, e.g. `mongo:7`.

NOTE Publishing `27017:27017` exposes the database on your machine's network interface. There is no authentication configured, so on a shared or public network that is an open database. The app does not need it published at all — only the backend talks to Mongo — so removing the `ports:` block (as the `redis` service sensibly does with `expose:`) would be safer.

expose **versus** ports

```
redis:
  expose:
    - 6379 # reachable by other containers ONLY
```

Contrast with `ports:` - `"27017:27017"`, which publishes to your machine as well. `expose` is the right choice for a backing service nothing outside the project should touch — and it is a good habit worth applying to MongoDB here too.

What is missing

```
volumes:
  data:
```

NOTE This declares a named volume called `data` that **no service uses** — MongoDB uses a bind mount instead. It is harmless leftover.

Also worth noticing: the frontend has no `depends_on`, and neither `depends_on` waits for readiness — only for the container to **start**. MongoDB takes a little while to accept connections, so on a cold start the backend will fail its first attempts. Unlike some Compose files, these services carry no `healthcheck` and no `restart: always` on the backend, so it will not retry itself.

```
docker compose up -d --build # build both images and start all four services
docker compose ps # see what is running
docker compose logs -f backend # follow the backend's logs
docker compose down # stop and remove (./backend/data survives)
```

PART III

The kubernetes/ Folder

Six manifests, eight objects: a `hostPath` PV and Claim, four Deployments with their Services, and the two `NodePorts` that let a browser in.

8. How to Read These Manifests

From the repo: [kubernetes/](#)

Six YAML files, eight Kubernetes objects. Before opening them one at a time, here is the map and the rules that apply to all of them.

One file, two objects — the --- separator

Four of the six files define **two** objects each. YAML allows several documents in one file, separated by a line containing exactly three dashes:

```
apiVersion: apps/v1
kind: Deployment
# ... the Deployment's fields ...
---
apiVersion: v1
kind: Service
# ... the Service's fields ...
```

`kubectl apply -f mongodb.yaml` creates **both**. This is a common and sensible convention: a Deployment and the Service in front of it always travel together, so keeping them in one file means you cannot apply one and forget the other.

File	Objects it creates
<code>persistentVolume.yaml</code>	1 — a PersistentVolume
<code>persistentVolumeClaim.yaml</code>	1 — a PersistentVolumeClaim
<code>mongodb.yaml</code>	2 — Deployment <code>mongo-deployment</code> + Service <code>mongo-service</code>
<code>redis.yaml</code>	2 — Deployment <code>redis-deployment</code> + Service <code>redis-service</code>
<code>backend.yaml</code>	2 — Deployment <code>backend-deployment</code> + Service <code>backend-service</code>
<code>frontend.yaml</code>	2 — Deployment <code>frontend-deployment</code> + Service <code>frontend-service</code>

TIP Each document needs its own `apiVersion` and `kind` — they are separate objects that happen to share a file. Check what a file will actually create, without applying it, using `kubectl apply -f mongodb.yaml --dry-run=client`.

The order they must be applied in

Kubernetes objects depend on each other, so you work from the bottom of the stack upwards:

```
kubectl create namespace wanderlust # 0. the namespace, first of all
kubectl config set-context --current --namespace wanderlust # make it the default

kubectl apply -f persistentVolume.yaml # 1. storage supply
kubectl apply -f persistentVolumeClaim.yaml # 2. storage demand
kubectl apply -f mongodb.yaml # 3. the database + its Service
kubectl apply -f redis.yaml # 4. the cache + its Service
kubectl apply -f backend.yaml # 5. the API, which needs both of the above
kubectl apply -f frontend.yaml # 6. the React app
```

TIP The repo's own README suggests waiting three or four minutes after applying `redis.yaml` before the backend. That is sound advice here, because — as you will see in chapters 12 and 13 — none of the Deployments define readiness probes, so Kubernetes has no way to know when MongoDB is genuinely accepting connections.

Everything lives in the `wanderlust` namespace

Every object sets `namespace: wanderlust`. Because it is written into the files, the namespace must exist first — no `manifest` creates it.

Once you have run the `set-context` command above, `wanderlust` is your default and no `kubectl` command in this Part needs a `-n` flag.

Two labels per tier do all the wiring

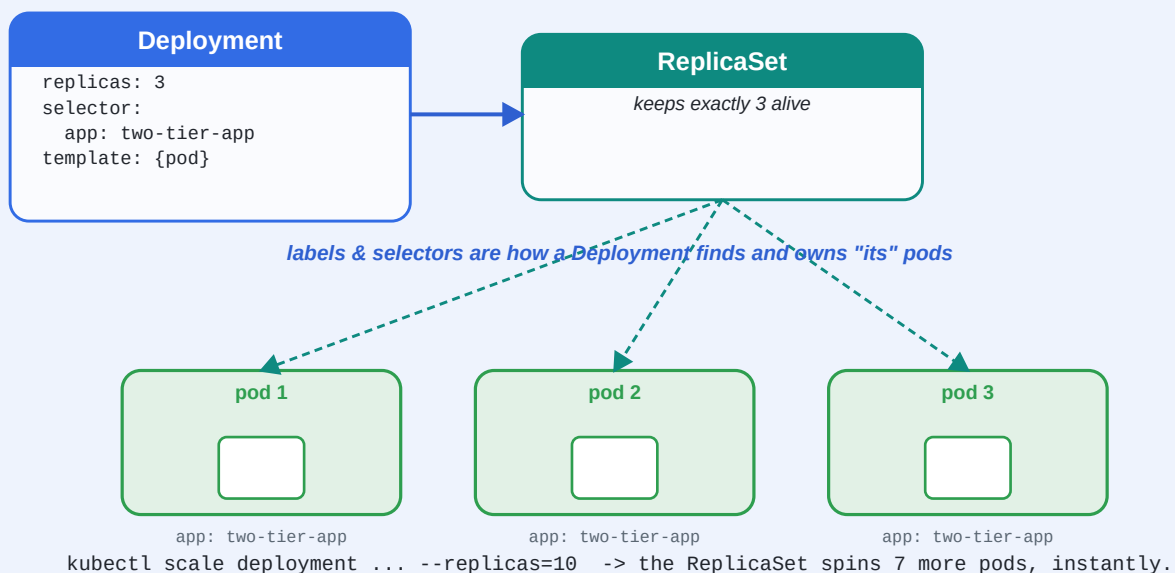
```
app: frontend # the React pods
app: backend  # the Express pods
app: mongo    # the MongoDB pod
app: redis    # the Redis pod
```

Nothing in this folder is connected by IP address. Every link — Deployment to pod, Service to pod — is a label match.

A shape that repeats

All four Deployments in this folder share the same skeleton, so once you have read `mongodb.yaml` carefully the others go quickly:

```
spec: # the DEPLOYMENT's spec: how many pods, which are mine
  replicas: 1
  selector:
    matchLabels: {...}
  template: # a pod BLUEPRINT starts here
    metadata:
      name: ... # (see the note in chapter 10 - this line does nothing)
      labels: {...} # the POD's labels
    spec: # the POD's spec: containers, volumes
      containers: [...]
```



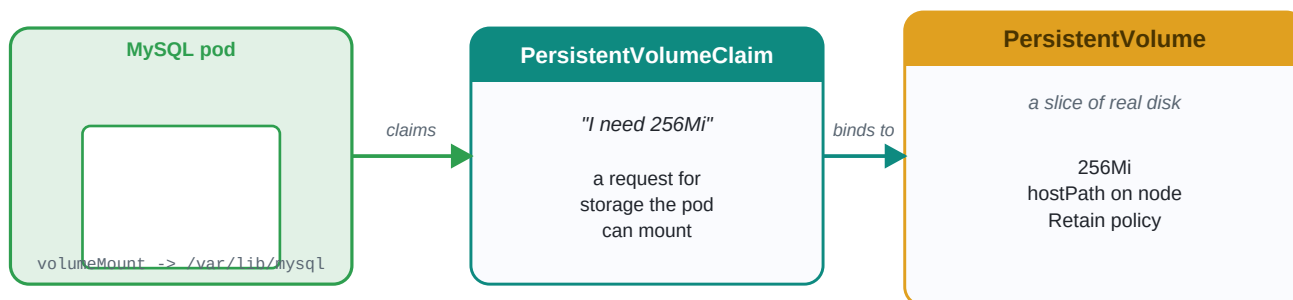
NOTE `spec:` appears twice and so does `metadata:`. That is not a typo. The outer `spec` is about **managing pods**; everything under `template:` is a complete pod definition that the Deployment stamps out. Edit anything under `template` and the Deployment rolls out replacements.

9. Storage — persistentVolume.yaml and persistentVolumeClaim.yaml

From the repo: [kubernetes/persistentVolume.yaml](https://kubernetes.io/docs/concepts/storage/persistentvolumes/)

These two short files solve one problem: **containers forget everything when they restart**.

A container's filesystem is temporary. Delete the pod and its files go with it — fine for a stateless API, unacceptable for a database. A **volume** is storage that outlives the pod.



Pods are disposable, so their data must live outside them. The pod claims a PVC, the PVC binds to a PV (real storage) — now MySQL's data survives a pod restart.

The supply side — persistentVolume.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongo-pv
  namespace: wanderlust
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/db
```

A **PersistentVolume (PV)** represents an actual piece of storage in the cluster — think of it as the disk itself.

Field	What it means
<code>capacity.storage: 5Gi</code>	Offers 5 gibibytes. Note Gi (1024-based), not G (1000-based).
<code>volumeMode: Filesystem</code>	Present it as a mountable directory rather than a raw block device — the usual choice.
<code>accessModes: [ReadWriteOnce]</code>	Mountable read-write by one node at a time.
<code>hostPath.path: /data/db</code>	Where the bytes physically live: that folder on whichever node the pod runs on.

NOTE A **PersistentVolume** is **cluster-scoped** — it does not belong to a namespace. The `namespace: wanderlust` line here is therefore ignored by Kubernetes. Harmless, but it is a copy-paste carry-over from the other files, and worth recognising so you do not go looking for a PV "inside" a namespace later.

WATCH OUT `hostPath` means "a folder on the node's own disk", and it is the least production-suitable volume type there is. If the pod is rescheduled onto a **different** node it finds an empty `/data/db` there and, as far as MongoDB is concerned, every post has vanished. If that node is terminated, the data is gone for good. It is fine for a single-worker learning cluster and nothing else. On a managed cluster you would use a CSI driver — the EBS CSI driver on EKS — to provision real network-attached storage.

The demand side — `persistentVolumeClaim.yaml`

From the repo: [kubernetes/persistentVolumeClaim.yaml](#)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
  namespace: wanderlust
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 5Gi
```

A **PersistentVolumeClaim (PVC)** is a *request* for storage. The pod asks for what it needs; Kubernetes finds a PV that satisfies it and **binds** the two together.

Field	What it means
<code>name: mongo-pvc</code>	The name the MongoDB Deployment references in its <code>volumes:</code> block
<code>accessModes: [ReadWriteOnce]</code>	Must be compatible with the PV's mode, or the two will never bind
<code>storageClassName: ""</code>	"Do not dynamically provision anything — bind me to an existing PV."
<code>resources.requests.storage: 5Gi</code>	Asks for 5 GiB. The PV offers exactly that, so it qualifies.

KEY IDEA PV and PVC are a **supply and demand** pair, like a job advert and an application. The PV says "5 GiB available here"; the PVC says "5 GiB wanted". Kubernetes plays matchmaker. The pod only ever names the **claim** — it never needs to know where the storage actually is, which is what lets you swap `hostPath` for a cloud disk without touching the Deployment.

`storageClassName: ""` is the detail that makes this work

This project gets right something many tutorials get wrong, so it deserves a moment.

The empty string is **not** the same as leaving the field out:

What you write	What Kubernetes does
<code>storageClassName: ""</code>	Disables dynamic provisioning — binds to an existing PV, here <code>mongo-pv</code>
<i>(field omitted)</i>	Uses the cluster's default StorageClass and provisions a brand-new volume, ignoring <code>mongo-pv</code> entirely

In the second case everything *appears* to work — the PVC reaches `Bound`, the pod starts, the app runs — but your data is somewhere you did not choose and `mongo-pv` sits unused at `Available` forever.

TIP Check which happened with `kubectl get pvc mongo-pvc` and look at the `VOLUME` column: it should read `mongo-pv`, not `pvc-` followed by a random UUID. `STATUS` should be `Bound`, not `Pending`.

NOTE One gap worth knowing about: this PV has no `persistentVolumeReclaimPolicy`. For a manually created PV the default is `Retain`, which is the safe choice — deleting the claim leaves the data alone and marks the PV `Released`. Stating it explicitly would be clearer.

10. mongodb.yaml

From the repo: [kubernetes/mongodb.yaml](https://github.com/wanderlust/k8s/blob/master/mongodb.yaml)

The database, and the first file with two objects in it. We will take this one slowly — the shape repeats three more times.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  namespace: wanderlust
  labels:
    app: mongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      name: mongo
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo
          ports:
            - containerPort: 27017
          volumeMounts:
            - name: mongo-storage
              mountPath: /data/db
      volumes:
        - name: mongo-storage
          persistentVolumeClaim:
            claimName: mongo-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
  namespace: wanderlust
spec:
```

```
selector:
  app: mongo
ports:
  - port: 27017
    targetPort: 27017
    protocol: TCP
```

replicas: 1 — and why it must stay 1

```
replicas: 1 # exactly one MongoDB pod
```

For the frontend you could happily run several copies. For this database you must not, for two reasons:

- 1 The PV is `ReadWriteOnce` — only one node can mount it at a time.
- 2 Two `mongo` processes writing the same data directory will corrupt it.

Scaling a database is not a matter of raising `replicas`; it needs a replica set, which in Kubernetes means a `StatefulSet` with per-pod storage rather than a `Deployment`.

NOTE A `Deployment` is designed for *stateless* workloads — interchangeable pods with no identity. Databases want stable names, stable storage and ordered startup, which is what a `StatefulSet` provides. Running MongoDB as a one-replica `Deployment` is fine for a demo, and being able to name that trade-off is a good answer when an interviewer asks how you would harden this.

selector and template.metadata.labels must match

```
selector:
  matchLabels:
    app: mongo # "the pods I manage wear this sticker"
template:
  metadata:
    labels:
      app: mongo # "...and here I am putting that sticker on them"
```

The `Deployment` finds its own pods by this label, and `mongo-service` below searches for the very same one.

WATCH OUT These two must match, and `selector` is **immutable** after creation. If you later want to rename the label you cannot edit it in place — you must delete the `Deployment` and recreate it. Kubernetes rejects the update with a validation error.

NOTE `template.metadata.name: mongo` does nothing. Pods created by a `Deployment` get generated names like `mongo-deployment-7d9f8b6c5-x2k4p`, so a `name` inside the pod template is ignored. Only the `labels` there matter. All four `Deployments` in this folder carry that same harmless leftover.

The container

```
- name: mongo
  image: mongo # no tag at all
  ports:
    - containerPort: 27017
```

`name: mongo` is an identifier inside the pod; you use it with `kubectl logs <pod> -c mongo` when a pod has several containers.

`containerPort: 27017` declares the port MongoDB listens on. Worth knowing: this **opens nothing** — the process is reachable on that port whether or not you declare it. The field is documentation for humans and tools.

WATCH OUT `image: mongo` has **no tag**, so Docker silently appends `:latest`. Two pods created a month apart can therefore run different major versions of MongoDB, and a major upgrade can refuse to open a data directory written by an older release — turning a routine pod restart into a database that will not start. Pin it: `image: mongo:7`.

NOTE There are no `MONGO_INITDB_ROOT_USERNAME` / `PASSWORD` variables here, so this MongoDB runs with **no authentication**. That matches `MONGODB_URI="mongodb://mongo-service/wanderlust"` in chapter 6, which carries no credentials. It is only tolerable because the Service is `ClusterIP`, so nothing outside the cluster can reach the database. Any real deployment needs credentials, held in a `Secret`.

volumeMounts and volumes — the storage handshake

These two blocks always come as a pair, and they live at different levels:

```
volumeMounts: # inside the CONTAINER: where to attach it
  - name: mongo-storage
    mountPath: /data/db # MongoDB's data directory
volumes: # inside the POD: what the volume actually is
  - name: mongo-storage # the same name links the two
    persistentVolumeClaim:
      claimName: mongo-pvc # the PVC from chapter 9
```

Read it as two sentences:

- 1 **volumes** (pod level): "This pod has a volume called `mongo-storage`; its contents come from the claim `mongo-pvc`."
- 2 **volumeMounts** (container level): "Attach the volume called `mongo-storage` at `/data/db` inside this container."

The shared string `mongo-storage` joins them, and it is purely local to this pod — it has nothing to do with the PV's or PVC's names. The full chain is:

```
mongod writes to /data/db -> mongo-storage -> mongo-pvc (PVC) -> mongo-pv (PV) -> /data/db on the node
```

KEY IDEA Storage in Kubernetes is deliberately indirect, and each hop earns its place. The container knows only a mount path. The pod knows only a claim name. The claim knows only a size and an access mode. Only the PV knows where the bytes really are.

The Service

```

apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  selector:
    app: mongo
  ports:
    - port: 27017
      targetPort: 27017
      protocol: TCP

```

Pods are disposable, and every restart gives them a **new IP address**. If the backend hard-coded the MongoDB pod's IP it would break the first time that pod was rescheduled. A **Service** solves this with a fixed name and virtual IP.

Three types of Service

ClusterIP	NodePort	LoadBalancer
default	30000-32767	cloud only
Reachable ONLY inside the cluster.	Opens a port on every node's IP.	Cloud provisions a real load balancer + URL.
Use it for the database tier (pod-to-pod).	Reach the app at NodeIP:30007.	Best on EKS / managed Kubernetes.

Field	What it means
<code>name: mongo-service</code>	Becomes a DNS name inside the cluster. This exact string is what <code>MONGODB_URI</code> refers to.
<code>selector.app: mongo</code>	Forwards to every pod labelled <code>app: mongo</code>
<code>port: 27017</code>	The port the Service listens on
<code>targetPort: 27017</code>	The port on the pod to forward to

This Service specifies no `type`, so it gets the default: **ClusterIP** — reachable only from inside the cluster. Exactly right for a database.

KEY IDEA The name of this Service is not cosmetic — it is the connection string. Rename it to `mongodb` and the backend breaks instantly, because `MONGODB_URI` in the image still says `mongo-service`. The manifest and the baked-in `.env.docker` are coupled by that one word, and the repo's README tells you to check `mongodb.yaml` for the name before building the image for exactly this reason.

TIP To prove a Service is wired up, run `kubectl get endpoints mongo-service`. If you see a pod IP and port, the selector matched. If you see `<none>`, it matched nothing.

11. redis.yaml

From the repo: [kubernetes/redis.yaml](https://github.com/wanderlust/k8s-manifests/blob/master/redis.yaml)

The cache. Structurally a twin of [mongodb.yaml](#), and the file with the most interesting history in the repository — it used to carry a volume block that could stop the pod from starting at all, and this chapter explains both the current file and what was removed.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-deployment
  namespace: wanderlust
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      name: redis
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis
          ports:
            - containerPort: 6379
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: wanderlust
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
      protocol: TCP
```

What Redis is doing here

Redis is an in-memory key/value store used as a **cache**. The backend's [utils/cache-posts.js](#) keeps recently-read posts in it, so repeated page loads do not hit MongoDB every time.

Crucially, the backend treats it as optional. From [backend/services/redis.js](#):

```
export async function connectToRedis() {
  try {
    if (REDIS_URL) {
      redis = await createClient({ url: REDIS_URL, disableOfflineQueue: true }).connect();
      console.log('Redis Connected: ' + REDIS_URL);
    } else {
      console.log('Redis not configured, cache disabled.');
```

If the connection fails the error is logged and the app keeps going — slower, but working.

TIP That is a useful pattern to recognise: a **cache should be optional**. If losing your cache takes your whole site down, it is not a cache, it is a dependency. This code gets it right.

Notice what is not here: no volumes

Compare this Deployment with [mongodb.yaml](#) and the difference is a missing pair of blocks. MongoDB has [volumeMounts](#) and [volumes](#); Redis has neither. That is deliberate, and it is the right shape for a cache — but it was not always this way.

Earlier versions of this file carried a volume block copied from [mongodb.yaml](#) and never adjusted:

```
volumeMounts:
  - name: mongo-storage
    mountPath: /var/lib/redis/dump.rdb
volumes:
  - name: mongo-storage
    persistentVolumeClaim:
      claimName: mongo-pvc # <- the SAME claim MongoDB uses
```

It has since been removed, and it is worth understanding why — because each of its three problems is a mistake you could easily make yourself.

WATCH OUT 1. It shared MongoDB's claim. [mongo-pvc](#) is already mounted by the MongoDB pod, and it is [ReadWriteOnce](#) — mountable read-write **by one node at a time**. If both pods happened to land on the same node it worked; if the scheduler put them on different nodes, the second pod stuck in [ContainerCreating](#) forever with a [FailedAttachVolume](#) event. On a two-node cluster that is a coin toss, which is the worst kind of bug: it passes the first time you try it.

WATCH OUT 2. Redis would write into MongoDB's data directory. The claim is backed by [hostPath: /data/db](#). Redis snapshots landing among MongoDB's files is, at best, confusing.

WATCH OUT 3. The mount path named a file, not a directory. A volume always mounts as a **directory**, so Kubernetes created [/var/lib/redis/dump.rdb](#) as a folder. Redis's default working directory is [/data](#) and it writes its snapshot to [/data/dump.rdb](#), so the mount was not where Redis saves anything — it was simply an odd empty directory inside the container.

Deleting both blocks fixes all three at once, and costs nothing: **a cache does not need persistence**. That is the point of a cache — everything in it can be rebuilt from MongoDB on demand, which is exactly what [cache-posts.js](#) does on a miss.

If you *did* want Redis snapshots to survive a restart, it would need **its own** PVC and a mount at `/data`, never a shared one:

```
volumeMounts:
  - name: redis-data # its OWN volume, not MongoDB's
    mountPath: /data # where Redis actually writes dump.rdb
volumes:
  - name: redis-data
    persistentVolumeClaim:
      claimName: redis-pvc # a separate claim, separately provisioned
```

KEY IDEA When you copy a manifest to make a new one, the volume block is the most dangerous part to leave unedited. Names like `mongo-storage` and `mongo-pvc` inside a Redis Deployment are a strong smell — and here they pointed at a real scheduling hazard. Read every copied block and ask what it means *for the new workload*.

The rest of the file

```
image: redis # again: no tag, so ":latest"
```

Same unpinned-tag issue as MongoDB. `redis:7-alpine` would be better — pinned, and a fraction of the size. Interestingly, `docker-compose.yml` pins `redis:7.0.5-alpine`; the manifest does not.

The Service is the same `ClusterIP` shape as `mongo-service`, and its name, `redis-service`, is what `REDIS_URL="redis://redis-service:6379"` in chapter 6 refers to.

TIP Because the pod now holds no volume, nothing ties it to a particular node. The scheduler can place it anywhere, and losing it costs only a cold cache for a few seconds. That is what a stateless workload should look like.

12. backend.yaml

From the repo: [kubernetes/backend.yaml](https://github.com/GoogleCloudPlatform/wanderlust/blob/master/backend.yaml)

The Express API, and the first object in this folder reachable from outside the cluster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: wanderlust
  labels:
    app: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      name: backend
      labels:
        app: backend
    spec:
```

```

containers:
  - name: backend
    image: <your-dockerhub-username>/wanderlust-backend-beta:v2.1.0
    ports:
      - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: wanderlust
spec:
  type: NodePort
  selector:
    app: backend
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
      nodePort: 31100

```

Notice what is not here

Compare this Deployment with [mongodb.yaml](#) and the striking thing is how short it is. There is:

- no `env`: block,
- no `envFrom`: pointing at a ConfigMap or Secret,
- no volumes.

So where does the backend get `MONGODB_URI`, `REDIS_URL`, `PORT` and `JWT_SECRET`? From the `.env.docker` file **baked into the image** by chapter 4's `COPY .env.docker .env`.

KEY IDEA This is the consequence of that design choice, and now you can see its full cost. Changing the database hostname, the port, or the JWT secret means **rebuilding and re-pushing the image** — you cannot change them by editing a manifest. It also means the same image cannot serve two environments, and that your secret ships inside a layer anyone with the image can read. The Kubernetes-native alternative is to remove `COPY .env.docker .env` from the Dockerfile and put the values here instead: `yaml envFrom: - configMapRef: name: backend-config # MONGODB_URI, REDIS_URL, PORT - secretRef: name: backend-secret # JWT_SECRET``

The image placeholder

```
image: <your-dockerhub-username>/wanderlust-backend-beta:v2.1.0
```

`<your-dockerhub-username>` must be replaced with your real Docker Hub account before this will pull.

WATCH OUT The tag here is `v2.1.0`, but the repo's own README builds and pushes `v2.1.8` — and under a different repository name (`backend-wanderlust` rather than `wanderlust-backend-beta`). Whatever you actually build and push, the `image`: line must match it **exactly**, in both this file and `frontend.yaml`. A mismatch gives you `ErrImagePull` with a `not found` message.

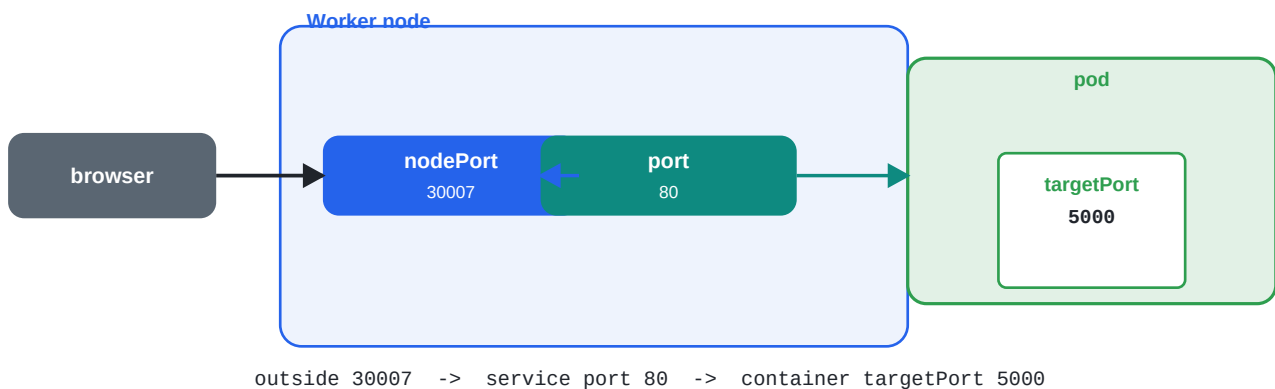
TIP Unlike [mongo](#) and [redis](#) above, this image **is** pinned to a version — which is good practice. A fixed tag also means [imagePullPolicy](#) defaults to [IfNotPresent](#), so the node reuses a cached copy. If you rebuild and re-push the *same* tag, the node will keep serving the old image; either bump the tag or run `kubectl rollout restart deployment/backend-deployment`.

The Service — a NodePort, and three port numbers

```
type: NodePort
ports:
- port: 8080
  targetPort: 8080
  protocol: TCP
  nodePort: 31100
```

This is the first Service in the guide with **three** port numbers, and confusing them is a rite of passage. Read from the outside in:

How a NodePort Service maps its three ports



Field	Value here	Meaning
<code>nodePort</code>	<code>31100</code>	The port opened on every node's own IP , reachable from outside the cluster
<code>port</code>	<code>8080</code>	The port the Service itself listens on, inside the cluster
<code>targetPort</code>	<code>8080</code>	The port on the pod — where Express is actually listening

So the journey is:

```
browser -> <worker-node-ip>:31100 -> Service :8080 -> pod :8080 -> Express
```

NOTE The figure above shows the generic pattern with different numbers; this project's actual values are the ones in the table — `31100`, `8080`, `8080`.

KEY IDEA `targetPort: 8080` must match `containerPort: 8080`, which must match `PORT=8080` in `backend/.env.docker`. Three files, one number — and if `PORT` were missing, `server.js` would fall back to 5000 and the Service would forward into silence. When a page loads but no data appears, walking that chain is the fastest way to find the break.

Why the API is exposed at all

You might reasonably ask why the backend needs a NodePort when the frontend could proxy to it internally. The answer is chapter 6's: the React code runs **in the user's browser**, outside the cluster, so it needs a publicly reachable address for the API. `VITE_API_PATH` points at `http://<worker-node-public-ip>:31100` — this Service.

WATCH OUT `nodePort` must fall within `30000–32767` by default, and `31100` is valid. Opening the port on the node is not always enough, though: on an EC2 worker you must also allow inbound TCP **31100** (and **31000** for the frontend) in the **security group**, or the connection simply times out with no Kubernetes error at all. This is the most common reason a correctly-deployed NodePort app appears unreachable.

What is missing: health probes and resource limits

This Deployment defines no `livenessProbe`, no `readinessProbe` and no `resources`. Both are worth adding, and the backend even has a suitable endpoint already — `server.js` answers `GET /` with a plain-text greeting:

```
readinessProbe: # "can this pod serve requests right now?"
  httpGet:
    path: /
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
resources:
  requests: # what the scheduler reserves for this pod
    memory: "128Mi"
    cpu: "100m"
  limits: # the ceiling it may not exceed
    memory: "512Mi"
    cpu: "500m"
```

NOTE A **readiness** probe failing removes the pod from its Service's endpoints — no restart, traffic simply goes elsewhere until it recovers. A **liveness** probe failing **restarts the container**. Always define readiness; add liveness only when you have a genuine hang scenario, and give it a generous `initialDelaySeconds` or a slow-starting pod will be killed in a loop.

13. frontend.yaml

From the repo: [kubernetes/frontend.yaml](https://github.com/kubernetes/frontend.yaml)

The last manifest, and the one users actually visit.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
  namespace: wanderlust
  labels:
    app: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
```

```

    app: frontend
  template:
    metadata:
      name: frontend
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: <your-dockerhub-username>/wanderlust-frontend-beta:v2.1.0
          ports:
            - containerPort: 5173
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: wanderlust
spec:
  selector:
    app: frontend
  type: NodePort
  ports:
    - port: 5173
      targetPort: 5173
      protocol: TCP
      nodePort: 31000

```

By now this holds no surprises — it is `backend.yaml` with different numbers:

	<code>backend.yaml</code>	<code>frontend.yaml</code>
Deployment	<code>backend-deployment</code>	<code>frontend-deployment</code>
Label	<code>app: backend</code>	<code>app: frontend</code>
Container port	<code>8080</code>	<code>5173</code>
Service type	<code>NodePort</code>	<code>NodePort</code>
<code>nodePort</code>	<code>31100</code>	<code>31000</code>
Visited by	The user's browser, via <code>VITE_API_PATH</code>	The user's browser, directly

`5173` is Vite's default development-server port, which is why it appears here, in `containerPort`, in `EXPOSE 5173`, and in the `--host` flag's server — all tracing back to chapter 5.

The two NodePorts, and how they pair up

This is the piece that ties Parts II and III together:

```

http://<worker-node-ip>:31000 -> frontend-service -> React pod :5173 (the page)
http://<worker-node-ip>:31100 -> backend-service -> Express pod :8080 (the API the page
calls)

```

The browser fetches the page from **31000**, and the JavaScript it receives then calls **31100** — because `frontend/.env.docker` was built with `VITE_API_PATH="http://<worker-node-public-ip>:31100"`.

WATCH OUT Those two numbers must agree, and they live in different places: `31100` is set in `kubernetes/backend.yaml` but consumed from an image built out of `frontend/.env.docker`. Change the backend's `nodePort` without rebuilding the frontend image and the site loads but stays empty. This is the most common failure in the whole project, and the only visible symptom is a failed request in the browser's developer console.

replicas: 1 — but here you could raise it

Unlike MongoDB, the frontend is **stateless**: it serves the same files to everyone and keeps nothing between requests. So `kubectl scale deployment/frontend-deployment --replicas=3` would genuinely work, and the Service would load-balance across all three. The same is true of the backend.

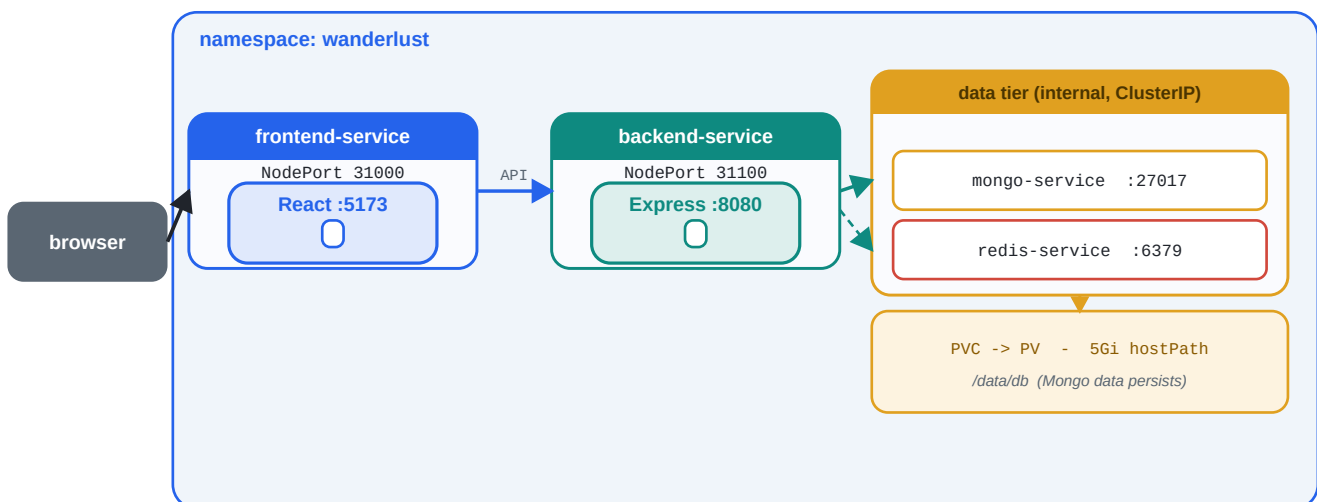
That is the practical difference between the tiers, and a good thing to be able to explain: stateless tiers scale horizontally by changing one number; stateful ones need far more care.

14. Putting It All Together

You have now read all six manifests. Here is the whole journey of a single page view, using only the objects you have seen.

The MERN app on a kubernetes cluster

React frontend (NodePort 31000) -> Express backend (NodePort 31100) -> MongoDB & Redis (internal)



A user opens the blog

- 1 The user browses to `http://<worker-node-ip>:31000`. That port is open on the node because `frontend-service` is a **NodePort**.
- 2 The Service forwards to port 5173 on a pod matching `app: frontend`.
- 3 Vite serves the React application. The browser downloads it and starts running it.
- 4 The JavaScript calls `http://<worker-node-ip>:31100/api/posts` — the value of `VITE_API_PATH`, baked in at image build time.
- 5 That request hits `backend-service` on nodePort 31100, which forwards to port 8080 on a pod matching `app: backend`.
- 6 Express checks Redis first, reaching it at `redis://redis-service:6379`. `redis-service` forwards to the pod matching `app: redis`.
- 7 On a cache miss it queries MongoDB at `mongodb://mongo-service/wanderlust`. `mongo-service` forwards to the pod matching `app: mongo`.
- 8 `mongod` reads from `/data/db`, which is the **PersistentVolumeClaim** `mongo-pvc`, bound to the **PersistentVolume** `mongo-pv`. The posts survive a pod restart.

9 The response travels back the same way and the posts appear on the page.

Which file did what

Step	File that made it possible
Public address for the page	<code>frontend.yaml</code> (<code>nodePort: 31000</code>)
Public address for the API	<code>backend.yaml</code> (<code>nodePort: 31100</code>)
Stable internal name for the cache	<code>redis.yaml</code>
Stable internal name for the database	<code>mongodb.yaml</code>
Pods actually running	the four <code>Deployment</code> objects
Data surviving a restart	<code>persistentVolume.yaml</code> + <code>persistentVolumeClaim.yaml</code>
The addresses the app uses	the two <code>.env.docker</code> files, baked in by the Dockerfiles
The images the pods run	<code>backend/Dockerfile</code> and <code>frontend/Dockerfile</code>

Verify it yourself

```
kubectl get pods # all four should be Running and READY 1/1
kubectl get svc # two ClusterIP, two NodePort
kubectl get pvc # mongo-pvc should say "Bound" to "mongo-pv"
kubectl get endpoints # every service should list at least one pod IP
kubectl logs deployment/backend-deployment # look for "Database connected" and "Redis Connected"
kubectl get pods -o wide # which node each pod landed on - relevant to chapter 11
```

That backend log line — `Database connected: mongodb://mongo-service/wanderlust` — is printed by `config/db.js` and is the single best confirmation that the tiers found each other.

APPENDICES

Appendices

A full field reference, the placeholders you must replace, common errors decoded, interview questions, a glossary, and every link.

15. Field Reference Cheat Sheet

Every field used anywhere in this project, in one place.

Dockerfile instructions

Instruction	Runs at	Used here for
<code>FROM node:21 AS backend-builder</code>	build	A named build stage on the full Node image
<code>FROM node:21-slim</code>	build	The final, smaller stage
<code>WORKDIR /app</code>	build	The working folder for every later instruction
<code>COPY package*.json ./</code>	build	Dependency manifests alone, so the install layer caches (frontend)
<code>COPY . .</code>	build	The application source
<code>RUN npm i / npm install</code>	build	Install dependencies
<code>RUN npm run test</code>	build	Fail the build if tests fail
<code>COPY --from=<stage> /app .</code>	build	Copy out of an earlier stage
<code>COPY .env.docker .env</code>	build	Bake configuration into the image
<code>EXPOSE 8080</code>	build	Documentation only — opens nothing
<code>CMD ["npm", "start"]</code>	run	Start the app when a container starts

Docker Compose keys

Key	Purpose
<code>services:</code>	One block per container
<code>image:</code>	Pull a ready-made image
<code>build: ./backend</code>	Build from the Dockerfile in that folder
<code>container_name:</code>	Fixes the container's name, which is also its hostname on the network
<code>env_file:</code>	Load environment variables from a file
<code>ports: "host:container"</code>	Publish a port to your machine
<code>expose:</code>	Reachable by other containers only
<code>volumes: "host:container"</code>	A bind mount
<code>depends_on:</code>	Start order — not readiness
<code>restart: unless-stopped</code>	Restart unless you stopped it deliberately

Fields common to every manifest

Field	Purpose
<code>apiVersion</code>	The API group and version the object belongs to
<code>kind</code>	The object type
<code>metadata.name</code>	Its name, unique within its kind and namespace
<code>metadata.namespace</code>	The namespace it lives in — always wanderlust here
<code>metadata.labels</code>	Key/value stickers used by selectors
<code>spec</code>	The desired state
<code>---</code>	Separates two objects in one file

Deployment fields

Field	Purpose
<code>spec.replicas</code>	How many identical pods to keep running
<code>spec.selector.matchLabels</code>	Which pods this Deployment owns. Immutable after creation.
<code>spec.template.metadata.labels</code>	The labels stamped on each pod — must match the selector
<code>spec.template.metadata.name</code>	Ignored — pods get generated names
<code>spec.template.spec.containers[].name</code>	The container's name inside the pod
<code>spec.template.spec.containers[].image</code>	The image to pull
<code>spec.template.spec.containers[].ports.containerPort</code>	Documents the port the app listens on
<code>spec.template.spec.containers[].volumeMounts</code>	Where to attach a volume inside the container
<code>spec.template.spec.volumes</code>	What the pod's volumes actually are

Not used in this project, but worth adding:

Field	Purpose
<code>containers[].env / envFrom</code>	Environment variables, literal or from a ConfigMap/Secret
<code>containers[].readinessProbe</code>	Traffic check — failure removes the pod from Services
<code>containers[].livenessProbe</code>	Health check — failure restarts the container
<code>containers[].resources</code>	CPU and memory requests and limits
<code>spec.strategy.rollingUpdate</code>	How gradually new versions replace old ones

Service fields

Field	Purpose
<code>spec.type</code>	ClusterIP (default), NodePort or LoadBalancer
<code>spec.selector</code>	Which pods to send traffic to
<code>spec.ports[].port</code>	The port the Service listens on
<code>spec.ports[].targetPort</code>	The port on the pod
<code>spec.ports[].nodePort</code>	The port opened on every node, 30000–32767
<code>spec.ports[].protocol</code>	TCP (default), UDP or SCTP

Storage fields

Field	Purpose
PV <code>spec.capacity.storage</code>	How much storage this volume offers
PV <code>spec.volumeMode</code>	Filesystem (a directory) or Block (a raw device)
PV <code>spec.accessModes</code>	ReadWriteOnce , ReadOnlyMany or ReadWriteMany
PV <code>spec.hostPath.path</code>	A folder on the node — demo-only storage
PVC <code>spec.resources.requests.storage</code>	How much storage the pod wants
PVC <code>spec.storageClassName: ""</code>	Bind to an existing PV; do not dynamically provision

The ports, all in one place

Port	Where it appears	What it is
31000	<code>frontend.yaml</code> <code>nodePort</code>	The public page address
31100	<code>backend.yaml</code> <code>nodePort</code> , <code>VITE_API_PATH</code>	The public API address
5173	<code>frontend</code> Dockerfile, <code>containerPort</code> , <code>targetPort</code>	Vite's dev server
8080	<code>PORT</code> in <code>.env.docker</code> , <code>backend</code> Dockerfile, <code>containerPort</code> , <code>targetPort</code>	Express
27017	<code>mongodb.yaml</code> , <code>MONGODB_URI</code> (implied)	MongoDB
6379	<code>redis.yaml</code> , <code>REDIS_URL</code>	Redis
5000	<code>docker-compose.yaml</code> only	The <code>server.js</code> fallback — see chapter 7

16. Placeholders You Must Replace

Nothing will deploy until all of these are filled in.

Placeholder	Appears in	Replace with
<code><your-dockerhub-username></code>	<code>kubernetes/backend.yaml</code> , <code>kubernetes/frontend.yaml</code>	Your Docker Hub account name

Placeholder	Appears in	Replace with
<worker-node-public-ip>	frontend/.env.docker, backend/.env.docker	The public IP of the worker node you browse to
<your-strong-random-secret>	backend/.env.docker	Output of <code>openssl rand -hex 64</code>

Also check the **image tags**. The manifests say `v2.1.0`; the repo's README builds `v2.1.8` and uses different repository names. Whatever you build and push must match the `image:` lines exactly.

Find every remaining placeholder before you build:

```
grep -rn "<your-\"|<worker-node\" kubernetes/ backend/.env.docker frontend/.env.docker
```

And the order that matters: edit `frontend/.env.docker` **before** building the frontend image, and `backend/.env.docker` **before** building the backend image — both are baked in at build time.

WATCH OUT `frontend/.env.docker` is **not in the repository** — `frontend/.gitignore` ignores `.env*`. You have to create it yourself before the frontend image will build at all. See chapter 6.

17. Common Errors and What They Mean

The failures you are most likely to hit, and where the cause is explained.

What you see	What it means	Fix
namespaces "wanderlust" not found	The namespace does not exist yet	<code>kubectl create namespace wanderlust</code> (chapter 3)
ErrImagePull / ImagePullBackOff	The image name or tag does not exist	Replace <code><your-dockerhub-username></code> ; match the tag you pushed (chapter 12)
Pod Pending, PVC Pending	No PV matched the claim	Apply <code>persistentVolume.yaml</code> ; check sizes and access modes agree (chapter 9)
PVC Bound to pvc-<uuid>	A default StorageClass provisioned new storage instead	Keep <code>storageClassName: ""</code> (chapter 9)
A pod stuck in ContainerCreating	FailedAttachVolume — two pods share one ReadWriteOnce claim but landed on different nodes	Give each workload its own claim, or drop the volume if it does not need one (chapter 11)
Backend logs connection error	It cannot reach MongoDB	Check <code>MONGODB_URI</code> matches the Service name in <code>mongodb.yaml</code> (chapter 10)
Backend logs Error connecting to Redis	Redis is not up, or <code>REDIS_URL</code> is wrong	Harmless — the cache is optional (chapter 11)
Service exists but nothing responds	Selector matches no pod labels	<code>kubectl get endpoints</code> — <code><none></code> confirms it (chapter 8)
NodePort URL times out	The node's security group blocks the port	Allow inbound TCP 31000 and 31100 (chapter 12)
Page loads but no posts appear	<code>VITE_API_PATH</code> is wrong or the backend is unreachable	Check the browser console; rebuild the frontend image after changing it (chapter 6)

What you see	What it means	Fix
Login does not persist	Cookies are <code>secure: true / sameSite: none</code> , so browsers need HTTPS	Expected over plain HTTP (chapter 6)
Frontend build fails: <code>.env.docker not found</code>	The file is git-ignored, so a fresh clone does not have it	Create <code>frontend/.env.docker</code> with one <code>VITE_API_PATH</code> line (chapter 6)
<code>docker build</code> hangs on <code>npm run test</code>	Jest's <code>--watchAll</code> did not exit	The <code>globalTeardown</code> normally handles it; use <code>--watchAll=false</code> (chapter 4)
Compose: backend cannot reach the DB	<code>.env.docker</code> names Kubernetes Services, not Compose containers	Override the hostnames for Compose (chapter 7)
Compose: <code>localhost:5000</code> refuses	<code>PORT=8080</code> inside, but Compose publishes 5000	Publish <code>8080:8080</code> , or set <code>PORT=5000</code> (chapter 7)

The commands that diagnose almost everything:

```
kubectl describe pod <pod-name> # the Events list at the bottom names the real cause
kubectl logs <pod-name> # what the application itself printed
kubectl get endpoints # proves whether Services found their pods
kubectl get pods -o wide # which node each pod landed on
kubectl get pvc # Bound or Pending, and which volume it actually got
```

18. Interview Questions and Answers

Q. Why does the frontend Dockerfile copy package*.json before the source, while the backend copies everything first?

A. Because of Docker's layer cache. Dependencies change rarely; source code changes constantly. Copying the dependency manifests alone and installing them first means an ordinary code edit only invalidates the final `COPY . .` layer. The backend does `COPY . .` before `npm i`, so every code change invalidates the install layer and reinstalls everything — a two-second rebuild becomes a two-minute one.

Q. Both Dockerfiles are multi-stage. Does the backend actually benefit?

A. Barely. The technique is present — a `builder` stage on `node:21` and a final stage on `node:21-slim` — but the final stage runs `COPY --from=backend-builder /app .`, which copies the whole tree back, including `node_modules`, the tests and the coverage report. The only gain is the smaller base image. A real slimming would `npm ci --omit=dev` in the builder and copy just `node_modules` and the runtime source.

Q. What does `COPY --from=` do?

A. It copies files out of an earlier build stage instead of from your machine. It is what makes a multi-stage build worthwhile: you can use a large image full of compilers and test tooling to produce your application, then copy only the finished artefacts into a clean, small final image, leaving the tools behind.

Q. What is the difference between `RUN` and `CMD`?

A. `RUN` executes during `docker build` and its result is baked into the image as a layer. `CMD` executes when a container starts and sets the default command. `RUN npm i` puts libraries in the image; `CMD ["npm", "start"]` starts the server.

Q. Does `EXPOSE 8080` open a port?

A. No. It is documentation — metadata for humans and for `docker run -P`. The process is reachable on that port whether or not you declare it. What actually determines the port here is `PORT=8080` in `.env.docker`, read by `server.js`.

Q. Why does the frontend's CMD end with `-- --host`?

A. The `--` is npm syntax meaning "pass everything after this to the underlying command rather than to npm", so `--host` reaches Vite. There it means bind to `0.0.0.0` instead of `127.0.0.1`. Without it Vite would accept connections only from inside the container, and the port mapping would forward traffic nothing ever answers.

Q. What is wrong with shipping `npm run dev` as the production command?

A. It runs the Vite development server: slow to boot, holds the whole TypeScript toolchain in memory, serves unminified code, and is explicitly not intended for production. The production pattern is a multi-stage build that runs `npm run build` and copies the resulting `dist/` into an `nginx:alpine` image — around 25 MB instead of 1 GB.

Q. Why is `COPY .env.docker .env` in the Dockerfile a problem?

A. It bakes configuration — including `JWT_SECRET` — into the image. Anyone who can pull the image can read it. It also means one image per environment, because changing a hostname requires a rebuild and re-push. The Kubernetes answer is a ConfigMap for addresses and a Secret for the key, injected as environment variables at run time.

Q. Why does `VITE_API_PATH` use a public IP instead of `backend-service`?

A. Because the React code runs in the user's browser, outside the cluster, where internal DNS names mean nothing. It needs an address the browser can reach — the worker node's public IP on the backend's NodePort. The backend, running as a pod inside the cluster, can and should use the internal Service names `mongo-service` and `redis-service`.

Q. How does the backend find MongoDB?

A. Through `MONGODB_URI="mongodb://mongo-service/wanderlust"`. `mongo-service` is the name of the Kubernetes Service in `mongodb.yaml`, resolved by the cluster's internal DNS to the Service's virtual IP, which forwards to whichever pod currently matches `app: mongo`. No IP address appears anywhere, so the pod can be recreated freely.

Q. How do a Deployment and a Service find their pods?

A. Entirely through labels. The Deployment's `selector.matchLabels` identifies the pods it owns; a Service's `selector` identifies the pods it routes to. If a selector does not match, the Service has no endpoints and traffic silently goes nowhere, with no error message.

Q. What does the `---` in these YAML files do?

A. It separates two YAML documents in one file, so `mongodb.yaml` creates both a Deployment and a Service in a single `kubectl apply`. Each document needs its own `apiVersion` and `kind`. Keeping a Deployment and its Service together is a common convention so you cannot apply one and forget the other.

Q. Explain `port`, `targetPort` and `nodePort` in `backend.yaml`.

A. `nodePort: 31100` is the port opened on every node's IP, reachable from outside the cluster. `port: 8080` is the port the Service itself listens on inside the cluster. `targetPort: 8080` is the port on the pod, where Express listens. The path is browser to `<node-ip>:31100`, to Service `:8080`, to pod `:8080`.

Q. Why are MongoDB and Redis `ClusterIP` while the frontend and backend are `NodePort`?

A. Because only the two tiers a browser talks to need to be reachable from outside. `ClusterIP` is the default and makes the database and cache unreachable from the internet — a deliberate security boundary, which matters here because MongoDB runs with no authentication at all.

Q. How do a `PersistentVolume` and a `PersistentVolumeClaim` relate?

A. The PV is the supply — an actual piece of storage with a size and access mode. The PVC is the demand — a pod's request. Kubernetes binds a claim to a volume that satisfies it. The pod references only the claim, which is what lets you swap the underlying storage without editing the Deployment.

Q. What does `storageClassName: ""` mean, and why does it matter?

A. It explicitly disables dynamic provisioning, telling Kubernetes to bind to an existing PV rather than creating a new volume. Omitting the field entirely would instead use the cluster's default StorageClass and provision fresh storage, silently ignoring `mongo-pv` — everything would appear to work while the data sat somewhere you did not choose. This project gets it right.

Q. `redis.yaml` used to mount `mongo-pvc`. Why was that wrong, and why was deleting it the right fix?

A. Three reasons it was wrong. `mongo-pvc` is `ReadWriteOnce`, mountable by one node at a time, so if the Redis and MongoDB pods landed on different nodes the second stuck in `ContainerCreating` with `FailedAttachVolume`. It would have written Redis snapshots into MongoDB's data directory. And it mounted at `/var/lib/redis/dump.rdb`, a file path, where a volume always mounts as a directory — while Redis actually writes to `/data/dump.rdb`. Deleting both blocks fixes all three at no cost, because a cache does not need persistence: everything in it can be rebuilt from MongoDB on demand. If you did want it persisted, Redis would need its own PVC mounted at `/data`.

Q. Why is `image: mongo with no tag` a problem?

A. Docker silently appends `:latest`, which is a moving target. Two pods created a month apart can run different major versions, and a MongoDB major upgrade can refuse to open a data directory written by an older release — turning a routine pod restart into a database that will not start. Pin it to `mongo:7`.

Q. Why is MongoDB a Deployment with one replica rather than three?

A. Because its PersistentVolume is `ReadWriteOnce` — mountable by one node at a time — and two `mongod` processes writing the same data directory would corrupt it. Scaling a database needs a replica set, which in Kubernetes means a `StatefulSet` with per-pod storage. The frontend and backend, being stateless, genuinely can be scaled by changing `replicas`.

Q. Why is `hostPath` unsuitable for production?

A. It stores data in a folder on one specific node's local disk. If the pod is rescheduled to another node it finds an empty directory and the data appears to have vanished; if the node is terminated, the data is lost. On a managed cluster you would use a CSI driver — the EBS CSI driver on EKS — for network-attached storage.

Q. The backend Deployment sets no environment variables. How is it configured?

A. From the `.env.docker` file baked into the image by `COPY .env.docker .env` in the Dockerfile. That is why changing a database hostname or the JWT secret requires rebuilding and re-pushing the image rather than editing a manifest — and it is the main thing you would change if you were hardening this project.

Q. Why does `docker-compose.yml` not work with these `.env.docker` files?

A. Because those files are written for Kubernetes. They point at `mongo-service` and `redis-service`, which are Kubernetes Service names — under Compose the containers are called `mongo` and `redis`, so neither resolves. `PORT=8080` also disagrees with Compose publishing `5000:5000`. A single env file cannot serve two runtimes whose service names differ, which is exactly the problem ConfigMaps solve.

Q. What does a readiness probe do, and why does its absence matter here?

A. A readiness probe tells Kubernetes whether a pod can serve requests; if it fails, the pod is removed from its Service's endpoints without being restarted. None of these Deployments define one, so a pod is considered ready the moment its container starts — before Express has connected to MongoDB. That is why the repo's README tells you to wait several minutes between applying manifests: there is no automated way for the cluster to know.

19. Glossary

Term	Meaning
Base image	The image a Dockerfile builds on top of, named in <code>FROM</code>
Bind mount	A folder on the host mapped into a container, <code>host:container</code>

Term	Meaning
Build context	The folder sent to the Docker engine by <code>docker build</code> , filtered by <code>.dockerignore</code>
Cache (Redis)	A fast in-memory store holding results so the database is asked less often
ClusterIP	The default Service type — reachable only from inside the cluster
Container	A running instance of an image
Declarative	Describing the desired result rather than the steps to reach it
Deployment	A controller that keeps N identical pods alive and replaces them when they die
dotenv	The Node library that loads a <code>.env</code> file into <code>process.env</code>
Endpoint	The pod IP and port a Service currently forwards to
Express	The Node.js web framework serving this project's API
globalTeardown	A Jest hook that runs once after all test suites finish
hostPath	A volume backed by a folder on one node's own disk
Image	A read-only package containing an application and everything it needs
JWT	JSON Web Token — a signed token proving who a logged-in user is
kubeadm	The tool for building a self-managed Kubernetes cluster by hand
Label	A key/value sticker on an object, matched by selectors
Layer	A filesystem snapshot created by one Dockerfile instruction; the unit of Docker's cache
Manifest	A YAML file describing a Kubernetes object
MERN	MongoDB, Express, React, Node.js
Mongoose	The library the backend uses to talk to MongoDB
Multi-stage build	A Dockerfile with several <code>FROM</code> stages, keeping build tools out of the final image
Namespace	A folder inside the cluster grouping related objects — here <code>wanderlust</code>
NodePort	A Service type that opens a port <code>30000–32767</code> on every node
nodemon	A development tool that restarts a Node app when files change
Pod	The smallest deployable unit — one or more containers sharing a network address
PV	PersistentVolume — storage that exists independently of any pod
PVC	PersistentVolumeClaim — a request for storage, bound to a PV
Readiness probe	A check whose failure removes the pod from Service endpoints
Registry	A server that stores and serves images, such as Docker Hub
Replica	One of the identical pods a Deployment maintains
Selector	A label query determining which objects a controller or Service acts on
Service	A stable name and virtual IP in front of a changing set of pods

Term	Meaning
Service discovery	Finding another tier by name rather than by IP address
StatefulSet	The workload controller for stateful apps — stable names, storage and startup order
StorageClass	A named kind of storage a PVC can request for dynamic provisioning
Tag	The part of an image name after the colon, identifying a version
Vite	The build tool and dev server behind this project's React frontend

20. Links and Repository

The source code

All the code explained in this guide is in the author's GitHub repository. Every file quoted here is copied from it verbatim.

From the repo: github.com/nitin1094/wanderlust-mern-app-k8

What	Where
The repository	github.com/nitin1094/wanderlust-mern-app-k8
Backend Dockerfile	backend/Dockerfile
Frontend Dockerfile	frontend/Dockerfile
Compose file	docker-compose.yml
All Kubernetes manifests	kubernetes/
Deployment walkthrough	kubernetes/README.md
Building the cluster	kubernetes/kubeadm.md

Clone it and read along:

```
git clone https://github.com/nitin1094/wanderlust-mern-app-k8.git
cd wanderlust-mern-app-k8
```

Official documentation

Topic	Link
Dockerfile reference	docs.docker.com/reference/dockerfile
Dockerfile best practices	docs.docker.com/build/building/best-practices
Multi-stage builds	docs.docker.com/build/building/multi-stage
Docker Compose file reference	docs.docker.com/reference/compose-file
Kubernetes Deployments	kubernetes.io/docs/concepts/workloads/controllers/deployment
Kubernetes Services	kubernetes.io/docs/concepts/services-networking/service
Persistent Volumes	kubernetes.io/docs/concepts/storage/persistent-volumes
ConfigMaps	kubernetes.io/docs/concepts/configuration/configmap
Secrets	kubernetes.io/docs/concepts/configuration/secret

Topic	Link
Liveness and readiness probes	kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes
Creating a cluster with kubeadm	kubernetes.io/docs/setup/production-environment/tools/kubeadm
Vite — env variables and modes	vite.dev/guide/env-and-mode

Where to go next

- 1 Read the deployment walkthrough** — [kubernetes/README.md](#) in the repo takes you from two EC2 machines to a running app, with screenshots.
- 2 Harden the images** — add a [.dockerignore](#) to both folders, fix the backend's copy-then-install ordering, stop the test script asking for watch mode, and swap the frontend to a [vite build](#) plus nginx image.
- 3 Harden the manifests** — pin [mongo](#) and [redis](#) to real tags, add readiness probes and resource limits, and move configuration out of the images into a ConfigMap and a Secret. (The shared-PVC volume block in [redis.yaml](#) has already been removed — see chapter 11.)
- 4 Then scale it** — with configuration externalised and probes in place, raising [replicas](#) on the frontend and backend becomes safe, and MongoDB can move to a [StatefulSet](#) with real storage.